An algorithm for the multiple patterns approximate string matching problem

Chia-Wei Lu[1], Chun-Yuan Lin[2,*], Chuan-Yi Tang[1], Wing-Kai Hon[1]
[1]Department of Computer Science,
National Tsing Hua University, Hsinchu, Taiwan 300, ROC,
[2] Department of Computer Science,
Chang Gung University, Linkou Shiang, Taiwan 333, ROC
{d9762807@oz, wkhon@cs, cytang@cs}.nthu.edu.tw, cyulin@mail.cgu.edu.tw

Abstract

We propose an algorithm to solve the multiple patterns approximate string matching problem. Our algorithm uses a hashing table to quickly find all possible positions. Our algorithm is very efficient for multiple patterns with different lengths. We also implemented our algorithm to solve the short oligonucleotide alignment on a long reference DNA sequence problem. The experimental results show that our algorithm is faster than one of the fastest algorithm available now, SOAP.

1 Introduction

In this paper, we are concerned with the approximate string matching problem. Given two strings of characters, we usually use the edit distance to measure the similarity between them. The edit distance is defined as follows: Let A and B be two strings. The edit distance between A and B, denoted as $ED(A,B)$, is the minimum number of substitutions, insertions and deletions to transform B to A. A special version of the edit distance is the Hamming distance in which only substitutions are allowed.

The approximate string matching problem is defined as follows: We are given a text $T = t_1 t_2 \cdots t_n$, a pattern $P = p_1 p_2 \cdots p_m$ and a constant k. We are asked to find all of the substrings S_i's in T under the condition that $ED(S_i, P) \le k$. Comprehensive reviews of this problem can be found in [4,7-10,13,14,19,20]. The multiple patterns approximate string matching problem is a special version of the above problem in which a set of patterns, instead of one pattern, is given. That is, we have $PA_1, PA_2, \cdots, PA_N$ and we have to find all occurrences of PA_i's in T with edit distance less than or equal to k.

* The corresponding author.

In this paper, we shall first introduce an approximate string matching algorithm, namely the Navarro and Baeza-Yates Algorithm [14]. This algorithm is under the assumption that there is only one pattern. We shall later present our algorithm which is based upon the Navarro and Baeza-Yates Algorithm.

2 The Navarro and Baeza-Yates Algorithm

The Navarro and Baeza-Yates Algorithm is used to solve the approximate string matching with one pattern. The algorithm is based upon the following lemma:

Lemma 1 (see [14]) Let $P = P_1 X_1 P_2 X_2 ... P_j X_j$ for any $j \ge 1$, where P_i and X_i are substrings of P. Suppose there is a substring S of T whose edit distance from P is smaller than or equal to k. Then at least one P_i appears in S with at most $\lfloor k/j \rfloor$ errors.

To apply this algorithm, we may set j to be $k+1$. According to Lemma 1, the approximate string matching problem would first have to solve an exact string matching problem because we need to check the positions where some P_i appears with $\lfloor k/(k+1) \rfloor = 0$ error. Suppose at t_a, there is an exact matching of some $P_b = p_x p_{x+1} \cdots p_{x+|P_b|-1}$, we then open a window of T from $t_{a-x+1-k}$ to $t_{a-x+m+k}$ and check whether there is a substring S_i in this window such that $ED(S_i, P) \le k$. If at t_a, there is no exact matching of any P_i, we do not have to do any checking related to this location.

Thus there are two problems: (1) How to solve the exact matching problem? (2) How to find a substring within a window in T whose edit distance with respect to P is not larger than k. For the first problem, consult [1,2,3] and for the

second problem, consult [12,16,17].

The Navarro and Baeza-Yates Algorithm is a position elimination method. It effectively eliminates all positions which need not to be checked according to the algorithm. Nearly all of the approximate string matching problem, one way or the other, adopts this approach, except the Wu and Manber Algorithm [20].

For the single pattern approximate string matching problem, many algorithms would do some preprocessing on the pattern for having an efficiently alignment. But when we are concerned with the problem with multiple patterns, we may do some preprocessing on T. In the next section, we will introduce our algorithm to solve the multiple patterns approximate string matching problem based upon Lemma 1.

3 Our Algorithms

In our algorithm, we use the idea of Lemma 1 and set j to be $k+1$. Then we check the positions where some P_i appears with $\lfloor k/(k+1) \rfloor = 0$ error. That is, when $k = 2$, we will divide P into three parts $P_1P_2P_3$, and then only the positions of T where some P_i exactly appears are needed to check. Thus, we need to know where P_i appears in T with 0 error as fast as better. This is an exact string matching problem.

Many algorithms are based upon Lemma 1 [13,14,20]. It is noted that all of the algorithms open a window and then solve the exact matching problem without preprocessing the text. For example, the Sunday Algorithm [18] was used in [13].

But, we are dealing with the multiple patterns problem. That is, we are given a set of patterns. Therefore, we adopt the approach of conducting a preprocessing of the text T. After the preprocessing of the text is done, we can test the whole set of patterns quite easily, as we show later.

A typical method of preprocessing the text is to construct a suffix tree of T [5]. In this paper, we do not use the suffix approach. Instead, we use a hashing approach [15].

To simplify our discussion, let us start with the case of having only one pattern P. In this case, we divide the pattern into $k+1$ substrings with equal size. For example, suppose $k = 2$, and the length of P is 6. Then we divide P into 3 substrings with length 2.

The preprocessing of the text T is to find all substrings in T with length $\lfloor |P|/(k+1) \rfloor$. For example, in the above case, $\lfloor |P|/(k+1) \rfloor = \lfloor 6/(2+1) \rfloor = 2$. Let $T = a\ a\ c\ g\ a\ a\ g\ a\ g\ t\ t\ a$. Our preprocessing of T divides T into substrings with length equal to 2. The following are all distinct substrings which occur in T with length 2: *aa*, *ac*, *cg*, *ga*, *ag*, *gt*, *tt* and *ta*.

We further record the positions where each substring with length 2 occurs in T. For instance, *ac* occurs in position 2. The table *Ttable* in Fig. 1 illustrates our case.

Ttable

(*aa*) 0	1, 5	(*ga*) 8	4, 7
(*ac*) 1	2	(*gc*) 9	
(*ag*) 2	6, 8	(*gg*) 10	
(*at*) 3		(*gt*) 11	9
(*ca*) 4		(*ta*) 12	11
(*cc*) 5		(*tc*) 13	
(*cg*) 6	3	(*tg*) 14	
(*ct*) 7		(*tt*) 15	10

Fig. 1. An example of hashing table *Ttable*.

Note that the substrings of T are alphabetically ordered.

Suppose we have a pattern $P = a\ c\ a\ g\ t\ c$. The substrings after we divide the pattern into $k+1$ substrings are *ac*, *ag* and *tc*. We shall show that we can now use the hashing mechanism to conduct a quick search through the table and decide whether any substring of P occurs in T exactly. That we can use the hashing scheme is due to the fact that substrings of T and P are of equal length.

For a substring S of T or P, we first encode S to be a binary code S_{en}, $\lceil \log_2 |\Sigma| \rceil$ bits per character. Then, we use the decimal value $Val(S_{en})$ of S_{en} to be our hashing code. This hashing function is bijective. That is, $Val(S_{1en}) = Val(S_{2en})$ if and only if $S_1 = S_2$. For example, when $\Sigma = \{a, c, g, t\}$, we use 00, 01, 10 and 11 to substitute *a*, *c*, *g* and *t* respectively. For a substring $S = aacgt$, we have $S_{en} = 0000011011$ and $Val(S_{en}) = 0000011011_2 = 27$. Thus, the location of *aacgt* will be 27 in *Ttable*. *Ttable*[27] will contain the positions of T where *aacgt* occurs. For example, if *aacgt* occurs in positions 4 and 6 in T, then, *Ttable*[27] = {4, 6}.

For example, from the *Ttable*, using the hashing scheme, we will find that the hashing values of *ac* and *ag* are 1 and 2 respectively. From Fig. 1, by checking the locations of 1 and 2 in the *Ttable*, we know that *ac* occurs in position 2 of T and *ag* occurs in positions 6 and 8 in T. We will also know that *tc* does not occur in T at all.

According to Lemma 1, we only have to check the windows with size $|P|+2k=6+4=10$, starting from positions $(2-k)=0$, $(6-2-k)=2$ and $(8-2-k)=4$.

We can see that once the table *Ttable* of T is constructed and if the patterns are all of the same length, the exact matching problem for each pattern can be efficiently solved. In the following, we shall show that even if the lengths of the patterns are different, our method is still efficient.

We now assume that we have more than one patterns PA_i's with different lengths. If we record the positions of substrings of T with different lengths in *Ttable*, it would need a large size of memory. For saving memory, we only record the positions of every substring of T with length $\lfloor PA_{min}/(k+1) \rfloor$ in *Ttable*, where PA_{min} is the length of the shortest pattern in the set of PA_i's. Then, we will present an algorithm to handle the PA_i's whose lengths are larger than PA_{min}.

Consider that we are processing a PA_i whose length is larger than PA_{min}. We also divide PA_i into $k+1$ substrings PA_{ij}'s with length $\lfloor |PA_i|/(k+1) \rfloor$ as shown in Fig. 2.

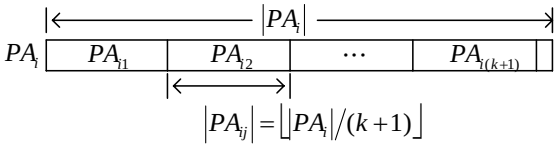

Fig. 2. The dividing of PA_i.

Note that we only construct the *Ttable* which records the positions of substrings of T with length $\lfloor PA_{min}/(k+1) \rfloor$. How could we use *Ttable* to find the PA_{ij}'s with lengths $\lfloor |PA_i|/(k+1) \rfloor$ in Fig. 2 which are larger than $\lfloor PA_{min}/(k+1) \rfloor$? Consider Fig. 3, we could use the occurrences of PA_{ij}^1 and PA_{ij}^2 to find the occurrences of PA_{ij}, where PA_{ij}^1 is a prefix of PA_{ij}, PA_{ij}^2 is a suffix of PA_{ij} and the lengths of PA_{ij}^1 and PA_{ij}^2 are both $\lfloor PA_{min}/(k+1) \rfloor$.

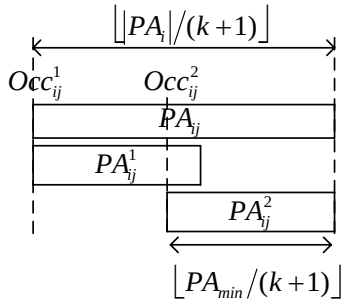

Fig. 3. Using PA_{ij}^1 and PA_{ij}^2 to obtain the occurrences of PA_{ij} in T when $|PA_{ij}| \le |PA_{ij}^1| + |PA_{ij}^2|$.

That is, let Occ_{ij}^1 and Occ_{ij}^2 be the sets of the occurrences of PA_{ij}^1 and PA_{ij}^2 in T respectively. Then the occurrences of PA_{ij} in T are the intersection of Occ_{ij}^1 and $Occ_{ij}^2 + \lfloor PA_{min}/(k+1) \rfloor - \lfloor |PA_i|/(k+1) \rfloor$ as shown in Fig. 3. Occ_{ij}^1 and Occ_{ij}^2 can be found from *Ttable*. The problem is: Could the intersection be done easily? The answer is yes because the elements of Occ_{ij}^1 and Occ_{ij}^2 are stored in an increasing order in *Ttable*. Thus, this step would not take much time. For instance, $T = a\ a\ c\ g\ a\ a\ g\ a\ g\ t\ t\ a$, $PA_2 = a\ a\ g\ g\ t\ c\ c\ t\ t$, $k=2$, and we have the *Ttable* as shown in Fig. 1. We now also divide PA_2 into 3 substrings, each with length 3, $PA_{21} = aag$, $PA_{22} = gtc$, and $PA_{23} = ctt$. From *Ttable*, we know that aa appears in positions $\{1,5\}$ and ag appears in positions $\{6,8\}$. Then, the intersection of the positions $\{1,5\}$ and $\{(6+2-3),(8+2-3)\} = \{5,7\}$ is $\{5\}$. Thus, we have $PA_{21} = aag$ appears in position 5 of T.

Let us consider the case when the length of PA_{ij} is larger than $|PA_{ij}^1| + |PA_{ij}^2|$, as shown in Fig. 4.

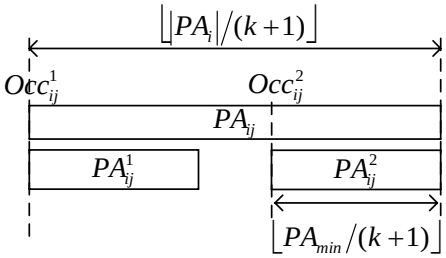

Fig. 4. Using PA_{ij}^1 and PA_{ij}^2 to obtain the occurrences of PA_{ij} in T when $|PA_{ij}| > |PA_{ij}^1| + |PA_{ij}^2|$.

As shown in Fig. 4, the intersection of the occurrences of PA_{ij}^1 and PA_{ij}^2 in T may contain some positions of substrings of T which are not PA_{ij}. Since we do checking of approximate matching any way, we would not miss any approximate occurrence of PA_i.

I. Preprocessing phase

In our preprocessing phase, we will construct *Ttable* which records the positions of every substring of T with length $\lfloor PA_{min}/(k+1) \rfloor$. Before the construction, we first transform T to a binary code form, $\lceil \log_2 |\Sigma| \rceil$ bits per character, denoted by T_{en}. Let $Val(S_{en})$ be the decimal value of the binary code of a substring S. Let $C = \lfloor PA_{min}/(k+1) \rfloor$ and n be the length of T. The total number of the substrings in T with

length C is therefore $(n-C+1)$. Let $T_{en}(i, j)$ be the binary code of the substring $t_i t_{i+1}...t_j$ of T. The algorithm for constructing *Ttable* is shown in Algorithm-A.

Algorithm-A (Preprocessing of Text *T*)
Input: T_{en} and C.
Output: Two dimensional array *Ttable*[*MaxRowIndx*][*MaxColumnIndx*].

1 Create an array $Num[0..(|\Sigma|^C - 1)] = 0$.
2 For $i = 0$ to $(|\Sigma|^C - 1)$, set $Pointer[i] = i$.
3 Set $MaxRowIndx = |\Sigma|^C$, and $MaxColumnIndx = \left((n-C+1)/|\Sigma|^C\right)+1$.
4 For $i = 1$ to $n-C+1$, do
5 i. Compute $Val = Val(T_{en}(i, i+C-1))$.
6 ii. Set $Ttable[Pointer[Val]][Num[Val]] = i$.
7 iii. Set $Num[Val] = Num[Val] + 1$.
8 iv. If $Num[Val] = MaxColumnIndx$, create a new row: set
$Ttable[Pointer[Val]][MaxColumnIndx] = MaxRowIndx$,
$Pointer[Val] = MaxRowIndx$,
$MaxRowIndx = MaxRowIndx + 1$, and $Num[Val] = 0$.
9 Return $Ttable[MaxRowIndx][MaxColumnIndx]$.

For a substring $T_{en}(i, i+C-1)$, we first compute the decimal value $Val = Val(T_{en}(i, i+C-1))$, and we record the position i in $Ttable[Val][x]$ for some x. That is, the row $Ttable[Val][]$ stores the positions of the substrings with decimal value Val. We set the size of one row to be $\left((n-C+1)/|\Sigma|^C\right)+1$, and if the number of substrings with decimal value Val is larger than the row size, we will create a new row for the storing those positions with decimal value Val. We use the array $Num[Val]$ to record the number of substrings with decimal value Val, and $Pointer[Val]$ to indicate the rows that the substrings with decimal value Val are now going to be stored. If some row R_1 is pointing to a new row, namely R_2, we will set the last element of R_1 to be the index of R_2.

The total memory cost of *Ttable* is

$$MaxRowIndx \times MaxColumnIndx$$
$$= MaxRowIndx \times \left((n-C+1)/|\Sigma|^C + 1\right).$$

Consider the worst case when $T = aa\cdots a$. Then all the substring with length C are $aa\cdots a$, and their decimal values are all 0. Since the decimal values of all the substrings in T are all 0, and the total number of substrings is $(n-C+1)$, we have to create $(|\Sigma|^C - 1)$ new rows to store the positions of these substrings. Therefore, in worst case, $MaxRowIndx = (|\Sigma|^C + |\Sigma|^C - 2)$, and our table will cost about $(2 \times n)$ in space. The table in worst case for the alphabet size $|\Sigma|$ equal to 4 is as shown in Fig.5.

If we set *MaxColumnIndx* to be a constant C_{MAX} which is a smaller value than $(n-C+1)/4^C$, then the size of *Ttable* would be $(n + C_{MAX} \cdot (4^C - 1))$ in worst case. Since in general the size of T is very large and C is a small value, we have $n > C_{MAX} \cdot (4^C - 1)$, and $(n + C_{MAX} \cdot (4^C - 1)) < (2 \times n)$. Therefore, for saving memory, we set C_{MAX} to be 5000 in our experiments when $n = 100{,}000{,}000$, $C = 5$, and $n > C_{MAX} \cdot (4^C - 1) \approx 5{,}000{,}000$.

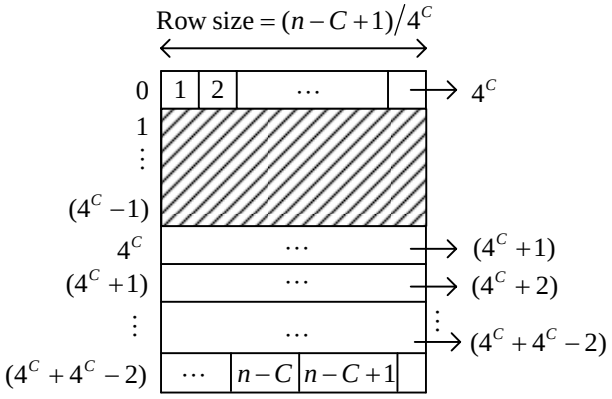

Fig. 5. The worst case of *Ttable* in space.

Since we process every substring $T_{en}(i, i+C-1)$ from $i = 1$ to $n-C+1$, the elements stored in one row will be in an increasing order, and this property allows us to find the intersection between two rows efficiently.

II. Searching phase

For a pattern PA_i, we divide PA_i into $k+1$ parts with length $\lfloor |PA_i|/(k+1) \rfloor$. That is

$$PA_i = PA_{i1} PA_{i2} ... PA_{i(k+1)} X_{i(k+1)}.$$

For each PA_{ij}, we compute the hashing code of PA_{ij}^1 and PA_{ij}^2, and let $Val_1 = Val(PA_{ij\,en}^1)$, $Val_2 = Val(PA_{ij\,en}^2)$. Let x be a position of T. Suppose x appears in $Ttable[Val_1][]$ and $(x + \lfloor |PA_i|/(k+1) \rfloor - C)$ appears in $Ttable[Val_2][]$. We then open a window W of T starting from $x - (j-1) \times C - k$ with length $|PA_i| + 2k$, and then we check whether there exists a substring of W whose edit distance from PA_i is smaller than or equal to

k within W. To do this checking, we may apply any approximation string matching algorithm suitable in this case [12,16,17].

Let $Val(PA_i(i, j))$ be the decimal value of substring $p_i p_{i+1} \ldots p_j$ of PA_i. Our algorithm for finding all occurrences of PA_i's in T with at most k errors is shown in Algorithm-B.

Algorithm-B (Searching Phase)
Input: PA_i's, T, C, k, $Ttable$ and $MaxColumnIndx$.
Output: All (PA_i, x, k')'s, where some PA_i appears in position x of T with $k' \le k$ errors.

1 For one PA_i, encode PA_i to the binary code form $PA_{i\,en}$.
2 For $j = 1$ to $k+1$ do
3 i. Compute $Val_1 = Val(PA^1_{ij\,en})$ and $Val_2 = Val(PA^2_{ij\,en})$.
4 ii. Set $Pointer_1 = Val_1$, and $Pointer_2 = Val_2$.
5 iii. Set $e_1 = 0$, $e_2 = 0$.
6 While($Ttable[Pointer_1][e_1] \neq 0$ & $Ttable[Pointer_2][e_2] \neq 0$) do
7 If $Ttable[Pointer_1][e_1] > Ttable[Pointer_2][e_2] + C - \lfloor |PA_i| / (k+1) \rfloor$, set $e_2 = e_2 + 1$.
8 Else if $Ttable[Pointer_1][e_1] < Ttable[Pointer_2][e_2] + C - \lfloor |PA_i| / (k+1) \rfloor$, set $e_1 = e_1 + 1$.
9 Else, check the window W which is starting from position $Pos = Ttable[Pointer_1][e_1] - (j-1) \times C - k$ with length $|PA_i| + 2k$, and if $k' = \mathrm{ED}(PA_i, W) \le k$, output ($PA_i$, Pos, k'). Set $e_1 = e_1 + 1$ and $e_2 = e_2 + 1$.
10 If $e_1 = MaxColumnIndx$, set $Pointer_1 = Ttable[Pointer_1][MaxColumnIndx]$, $e_1 = 0$, and if $e_2 = MaxColumnIndx$, set $Pointer_2 = Ttable[Pointer_2][MaxColumnIndx]$, $e_2 = 0$.
11 Repeat from Line 1 until all PA_i's are processed.

4 Applications and Experiments

In the bioinformatics field, there is a problem which is similar to ours. That is, given a long reference DNA sequence, a set of short oligonucleotides with different lengths and an error bound k, our job is to align these oligonucleotides onto the reference sequence with at most k errors. The error is measured by Hamming distance which is defined as the number of i's such that $a_i \neq b_i$ between two strings $A = a_1 a_2 \cdots a_m$ and $B = b_1 b_2 \cdots b_m$.

To solve this problem, we set T to be the long reference sequence and PA_i's to be the oligonucleotides. In addition, we modify Line 9 of our Algorithm-B by checking the Hamming distance between the window W which is starting from position $x = Ttable[Pointer_1][e_1] - (j-1) \times C$ with length $|PA_i|$ and the pattern PA_i. Computing the hamming distance between two strings could be done efficiently by XOR operations on substrings of T_{en} with $PA_{i\,en}$ and a lookup table.

Many algorithms have been proposed to solve this problem, and let us introduce one algorithm, called SOAP Algorithm [11] which is one of the fastest programs for this problem. The idea of SOAP Algorithm is to choose 4 non-overlapping substrings $PA_{i1}PA_{i2}PA_{i3}PA_{i4}$ of PA_i, each with length $\lfloor PA_{min}/4 \rfloor$. When $k = 2$, at least 2 of the substrings exactly match at the same time. That is, they need to know where $PA_{i1}PA_{i2}$, $PA_{i2}PA_{i3}$, $PA_{i3}PA_{i4}$, $PA_{i1}\_PA_{i3}$, $PA_{i2}\_PA_{i4}$ and $PA_{i1}\_\_PA_{i4}$ appears. Thus, they have to build 3 different hashing tables to do the exact matching part.

In our experiments, we took a part of chicken genome sequence with size 100Mb to be our T [6]. We used 580k different patterns with length $m = 15$~33 which are the output data of Illumina-Solexa system with eliminating adapter. We set error bound $k = 2$. We show the searching time of our algorithm in different lengths and the searching time of the SOAP Program (Usage: -s 7 -r 2 -n 1) as shown in Table 1. We also show the average searching time of PA_i's in different lengths in Fig. 6.

We also show the average of number of possible positions which are needed to check by our algorithm within different lengths of PA_i's in Table 2.

Table 1. The searching time of PA_i's in different lengths.

n=100M	Number of PA_i's	Our Algorithm (sec)	SOAP Algorithm (sec)
m=15~17	17,348	710	515
m=18~20	98,767	1,949	3,885
m=21~23	151,218	1,585	7,020
m=24~25	44,626	324	2,645
m=33	267,579	1,364	13,736
Total time		5,932	27,801

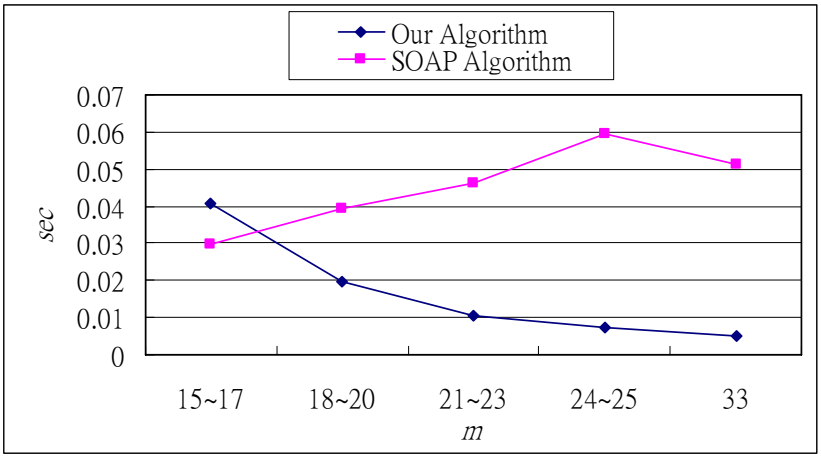

Fig. 6. The average searching time of one PA_i in different lengths.

Table 2. Number of positions needed to check by our algorithm for one PA_i within different lengths.

m	15~17	18~20	21~23	24~25	33
Number	291,620	86,622	26,633	11,147	2,167

Experimental results show that our algorithm is faster than SOAP Algorithm. For example, when the length is 33, our algorithm is faster than theirs by 10 times. Consider the results in Table 2, the larger PA_i is, the less possible positions are needed to check by our algorithm. This is why our algorithm is efficient especially for larger PA_i's. In addition, in this experiment, the working memory of our algorithm is about 430 Mb, and SOAP Program takes about 560 Mb.

5 Conclusion and Future Works

We have proposed an algorithm to solve the multiple patterns approximate string matching problem. Experimental results show that our algorithm takes less memory and is more efficient than the SOAP Algorithm.

In the future, we will try to compress our preprocessing table so that we can process exceedingly large sequences.

Reference

[1] M. Crochemore and W. Rytter, Text Algorithms, Oxford University Press, New York, 1994, 412 pages, ISBN 0-19-508609-0.

[2] M. Crochemore, C. Hancart and T. Lecroq, Algorithms on Strings, Cambridge University Press, 2007, 383 pages, ISBN 978-0-521-84899-2.

[3] M. Crochemore and W. Rytter, Jewels of Stringology, World Scientific, 2002, 310 pages, ISBN 981-02-4782-6.

[4] K. Fredriksson and G. Navarro, Average-Optimal Multiple Approximate String Matching, ACM Journal of Experimental Algorithmics, Vol. 9, No. 1.4, 2004, pp. 1-47.

[5] Daniel M. Gusfield, Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology, Cambridge University Press, 1997.

[6] Evgeny A. Glazov, Pauline A. Cottee, Wesley C. Barris, Robert J. Moore, Brian P. Dalrymple and Mark L. Tizard, A microRNA catalog of the developing chicken embryo identified by a deep sequencing approach, Genome Research, 2008, Vol. 18, pp. 957-964.

[7] H. Hyyrö, Bit-parallel approximate string matching algorithms with transposition, Journal of Discrete Algorithms, Vol. 3, 2005, pp. 215-229.

[8] T. N. D. Huynh, W. K. Hon, T. W. Lam and W. K. Sung, Approximate String Matching Using Compressed Suffix Arrays, Theoretical Computer Science, Vol. 352, 2006, pp. 240-249.

[9] J. Holub and B. Melichar, Approximate string matching using factor automata, Theoretical Computer Science, Vol. 249, 2000, pp. 305-311.

[10] H. Hyyrö and G. Navarro, Bit-parallel Witnesses and their Applications to Approximate String Matching, Algorithmica, Vol. 41, No. 3, 2005, pp. 203-231.

[11] Ruiqiang Li, Yingrui Li, Karsten Kristiansen and Jun Wang, SOAP: short oligonucleotide alignment program, BIOINFORMATICS APPLICATIONS NOTE, Vol. 24, no. 5, 2008, pp. 713–714.

[12] G. Landau and U. Vishkin, Fast Parallel and Serial Approximate String Matching, Journal of Algorithms, Vol. 10, 1989, pp. 157-169.

[13] G. Navarro and R. Baeza-Yates, Very fast and simple approximate string matching, Information Processing

Letters, Vol. 72, 1999, pp. 65-70.

[14] G. Navarro and R. Baeza-Yates, A Hybrid Indexing Method for Approximate String Matching, Journal of Discrete Algorithms, No. 1, Vol. 1, 2000, pp. 205-239.

[15] Z. Ning, A. J. Cox and J. C. Mullikin, SSAHA: a fast search method for large DNA databases, Genome Research, Vol. 11, 2001, pp. 1725–1729.

[16] S. B. Needleman and C. D. Wunsch, A general method applicable to the search for similarities in the aminoacid sequence of two proteins, Journal of Molecular Biology, Vol. 48, 1970, pp. 443-453.

[17] P. H. Sellers, String Matching with Errors, Journal of Algorithms, Vol. 20, No. 1, 1980, pp. 359-373.

[18] D. Sunday, A very fast substring search algorithm, Comm. of the ACM, Vol 33, No. 8, 1990, pp. 132–142.

[19] J. Tarhio and E. Ukkonen, Approximate Boyer-Moore String Matching, SIAM Journal on Computing, Vol. 22, No. 2, 1993, pp. 243-260.

[20] S. Wu and U. Manber, Fast Text Searching: Allowing Errors, Communications of the ACM, Vol. 35, 1992, pp. 83-91.